

Introduction to SQL

Part-1:

How to download and install Oracle 11-G

here G->Grid

1)Scaler functions

- a) String or Character functions
- b) Numeric functions
- c) Date functions
- d) Conversion functions

2)SQL statements in SQL/ORACLE

- a) DDL-Data definition language
- b) DML-Data manipulation language
- c) TCL-Transaction control
- d) DCL-Data control language

3)Constraints in SQL/ORACLE

- a) Unique
- b) Not null
- c) Check constraints
- d) Primary-key
- e) Foreign-key
- f) default

4)Normalization in ORACLE/SQL

Part-2: ***

- 1)Configure EMP (employee) and Dept(departmental) Tables
- 2)How to view records from the table
- 3)Operators available in SQL
- 4)Apply operators on table data

- 5)Distinct clause
- 6)Pattern matching (like or not like)
- 7)Display records using in operator (in or not in)
- 8)Between operator (between or not between)
- 9)Order by clause (sorting the records)
- 10)Aggregate function in SQL
- 11)Group by clause***
- 12)Having clause
- 13)Set operations in SQL
 - a) union
 - b) union all
 - c) interest
 - d) minus (-)
- 14)Sub query in SQL
- 15)Table alias in SQL
- 16)WGHO rule
- 17)co-related sub query
- 18)Exists clause (exists or not exists)
- 19)Joins***
 - a) Cartesian or cross join
 - b) inner join or Equi join
 - c) Left join or left outer join
 - d) Right join or right outer join
 - e) Full join or full outer join
 - f) self-join
- 20)Natural join
- 21)Implicit join

- 22)Explicit join
- 23)Perform join operation by applying clause
- 24)Perform join operation on more than 2 tables
- 25)ROW, NUM in ORACLE SQL
- 26)Inline queries in SQL ***
- 27)Row ID in SQL
- 28)Row comparison operators in SQL (any, all, sum)

Part-3:

Interview Questions

SQL:

It stands for structural query language; it is a non-procedural language and it is non case sensitive.

note:

Anything is enclosed in a ' ' (single quotes) that acts as a case sensitive

1)Scaler functions: Scaler function acts/returns only one value at a time these functions are also called as inbuilt functions.

Character or String functions: -These functions are used to manipulate the character or String

a) lower function: - This function converts the given upper-case character into lower case

note: *If one letter is there then it is-> char

*If more than one letter is there then it is-->String

example;

```
SQL> select lower('WELCOME') from dual;
```

```
LOWER ('
```

```
-----
```

```
o/p:  welcome
```

***How to display the user defined title name for the output?**

Ans:

```
SQL>select lower('WELCOME')"result" from dual;
```

```
RESULT
```

```
-----
```

```
o/p:  welcome
```

***How to display the user defined title name for the output by another method?**

Ans:

```
SQL>select lower ('WELCOME') as result from dual;
```

```
RESULT
```

```
-----
```

```
o/p:  welcome
```

note: If we use as result/anything it gives result/anything in upper case only.

***What is dual?**

Dual is a data dictionary table, it has only one column that is dummy, it's datatype is varchar. The main purpose of dual table is to execute scaler or inbuilt function.

or

Dual is a table automatically created by Oracle Database along with the data dictionary, it has one column DUMMY defined to be VARCHAR2(1), and contains one row with a value X.

b) Upper Function: -This function converts the given lower-case character into an upper case.

example;

```
* SQL> select upper('welcome') from dual;
Upper ('
-----
o/p:  WELCOME

* SQL>select upper('welcome') "result" from dual;
Result
-----
o/p:  WELCOME

* SQL>select upper('welcome') as result from dual;
RESULT
-----
o/p:  WELCOME
```

c) INITCAP: - This function is used to convert the first character of each word into an upper case

example;

```
* SQL>select initcap ('core java') from dual;

INITCAP ('
-----
o/p:  Core Java
```

d)Length function: -This function provides the number of character available in the given String

example;

```
* SQL>select length('testing') from dual;

LENGTH('TESTING')
-----
o/p:                7
```

e) substr /substring: - This function returns the substring from the given String based on the start and end positions.

example;

```
* SQL>select substr ('programming',4,4) from dual;
```

```
SUBS
```

```
-----
```

```
o/p:  gram
```

f) Sub functions: - These functions are also called as nested functions, here the inner function executes first, based on the result of the inner function the outer function executes.

example;

Query: From the given String welcome extract string come and find the number of character available there.

```
SQL> select length (substr ('welcome',4,4)) from dual;
```

```
LENGTH(SUBSTR('WELCOME',4,4))
```

```
-----
```

```
o/p: 4
```

* Here substr is the inner function and length is the outer function.

Query: The given string is Organization extract organ and display it in the upper case

```
SQL>select upper (substr ('organization',1,5) from dual;
```

```
UPPER
```

```
-----
```

```
ORGAN
```

f) ascii: - This function provides ascii value of the given character

example;

```
* SQL> select ascii('A') from dual;
```

```
ASCII('A')
```

```
-----
```

```
o/p: 65
```

```
* SQL> select ascii('a') from dual;
```

```
ASCII('A')
```

```
-----
```

```
o/p: 97
```

g) chr: -This function provides character from the given ascii value

example;

```
SQL> select chr (68) from dual;
```

```

      c
      ----
o/p:   D

```

h) Replace: - The replace function replaces the search string with another set of string or character

example;

```
SQL>select replace ('It is a Palace', 'is', 'was') from dual;
```

```

REPLACE ('ITISAP
-----
it was a palace

```

g) Translate: - This function translates the first set of character into another set of characters

example;

```
SQL> select translate ('abcdefgh', 'aceg', 'WXYZ') from dual;
TRANSLAT
-----
```

```
o/p:   WbXdYfZh
```

h) Lpad: - This function left padding to the string into the given string based on the length specified.

or

This function that helps in padding or adding a string to the left of the given string based on the length specified.

example;

```
* SQL> select lpad ('java',10,'#') from dual;
LPAD ('JAVA
-----
```

```
o/p: #####java
```

note: Here we specified length is 10 so it's padding the word from left

```
* SQL>select lpad ('java',4,'#') from dual;
LPAD ('JAVA
-----
```

```
o/p:   java
```

note: Here we specified length is 4 so it's doesn't pad the word bcz., the word length is 4 only.

i) Rpad: - This function right padding the string into the given string based on the length specified.

or

This function that helps in padding or adding a string to the right of the given string based on the length specified.

example;

```
* SQL>select rpad ('java',10,'#') from dual;
```

```
RPAD ('java
```

```
-----
```

```
o/p: java#####
```

```
* SQL>select rpad ('java,4,'#') from dual;
```

```
RPAD ('java
```

```
-----
```

```
o/p: java
```

j) Ltrim: -This function trims the set of character from left hand side, by default it trims blank spaces from left hand side.

example;

```
* SQL> select ltrim ('###java###','#') from dual;
```

```
LTRIM ('
```

```
-----
```

```
o/p: java###
```

```
* SQL> select ltrim ('XYZjavaXYZ','XYZ') from dual;
```

```
LTRIM ('
```

```
-----
```

```
o/p: javaXYZ
```

```
* SQL> select ltrim ('rangranavrama', 'rang') from dual;
```

```
LTRIM
```

```
-----
```

```
O/P: vrama
```

/*note: here the output comes vrama bcz we call rang to trim it checks the called characters before it encounters the false character i.e., rangrana comes under rang characters so. in rtrim also comes like this when we approach like this. */

note: for blank space:

example;

```
SQL> select ltrim (' java ') from dual;
```



```

LTRIM ('
-----
o/p: java

```

note: Here we using 3 blank spaces.

*if we need to confirm the blank is removed we need to check that using nested function i.e., example;

```

SQL>select length (ltrim (' java ')) from dual;

LENGTH(LTRIM('JAVA'))
-----
o/p:                4

```

k) Rtrim: - This function trims the set of character from right hand side, by default it trims blank spaces from right hand side.

example;

```

* SQL> select rtrim ('###java###','#') from dual;

RTRIM ('
-----
o/p: ###java

* SQL> select rtrim ('XYZjavaXYZ', 'XYZ') from dual;

RTRIM ('
-----
o/p: XYZjava

* SQL> select length (rtrim (' java ')) from dual;

LENGTH(RTRIM('JAVA'))
-----
o/p:                7

```

note: here it gives length as 7 in SQL space as also value and it countable

l) Trim: - This function trims the set of character from both the sides, it acts as super set on ltrim and rtrim.

example;

```

* SQL> select trim (leading '#' from'###java###') from dual;
TRIM (LE
-----
o/p: java###

```

```
* SQL>select trim (trailing '#' from'###java###') from dual;
TRIM (TR
-----
o/p: ###java
```

```
* SQL> Select trim (both '#' from'###java###') from dual;
TRIM
-----
o/p: java
```

Numeric Functions:

These functions are used to manipulate the number, decimals or integer values.

a) Absolute function: - This function provides absolute values of a given integer

example;

```
SQL> select abs (-10) from dual;

ABS (-10)
-----
o/p:      10
```

b) power: - This function provides m to the power of nth value (mⁿ)

example;

```
SQL> select power (2,5) from dual;

POWER (2,5)
-----
o/p:        32
```

c) Mod: - This function provides the remainder of the division

example;

```
* SQL> select mod (10,2) from dual;

MOD (10,2)
-----
o/p:        0
* SQL> select mod (11,2) from dual;

MOD (11,2)
-----
o/p:        1
```

d) Greatest: - This function provides the highest number from the list of given numbers

example;

```
SQL> select greatest (3,77,10,5,15) from dual;
```

```
GREATEST (3,77,10,5,15)
```

```
-----  
o/p:          77
```

e) least: - This function provides the lowest number from the list of given number

example;

```
SQL> select least (3,77,10,5,15) from dual;
```

```
LEAST (3,77,10,5,15)
```

```
-----  
o/p:          3
```

f) sqrt: - This function provides the square root of the given number

example;

```
SQL> select sqrt (25) from dual;
```

```
SQRT (25)
```

```
-----  
o/p:          5
```

g) Trunc: - This function trunc it the decimal positions based on the length of the decimal positions provides

example;

```
SQL> select trunc (12.56789,2) from dual;
```

```
TRUNC (12.56789,2)
```

```
-----  
o/p:          12.56          here it will trunc the decimal value given
```

h) round: - This function rounds the decimal positions based on the length provided

example;

```
SQL> select round (12.56789,2) from dual;
```

```
ROUND (12.56789,2)
```

```
-----
```

o/p: 12.57

here it will round the decimal to the next nearest value

i) floor: - This function provides the current integer value alone

example;

SQL> select floor (12.99999) from dual;

FLOOR (12.99999)

o/p: 12

j) ceil: - This function provides the next integer value

example;

* SQL> select ceil (12.00001) from dual;

CEIL (12.00001)

o/p: 13

* SQL> select ceil (12.99999) from dual;

CEIL (12.99999)

o/p: 13

Date Functions:

Date functions are used to manipulate the date value.

a) sysdate: - This function provides the current date based on the system date

or

This function provides the current system date.

*It provides the date DD-MON-YY format, it is a standard Oracle date format

example;

* SQL>select sysdate from dual;

SYSDATE

o/p: 17-SEP-23

*** We can add integer value in to a sysdate.**

yes, it increases the date value

example;

```
SQL>select sysdate+10 from dual;
```

```
SYSDATE+10
```

```
-----
```

```
o/p: 17-SEP-23
```

***Can we subtract integer value from sysdate function.**

yes, it subtracts the date value

example;

```
SQL>select sysdate-5 from dual;
```

```
SYSDATE-5
```

```
-----
```

```
o/p: 02-SEP-23
```

b) add_months: - This function allows to add the number of months into a given date.

example;

```
SQL>select add_months (sysdate,2) from dual;
```

```
ADD_MONTH
```

```
-----
```

```
o/p: 07-NOV-23
```

c) last_day: - This function provides the last date of the given date input.

example;

```
* SQL>select last_day (sysdate) from dual;
```

```
LAST_DAY (
```

```
-----
```

```
o/p: 30-SEP-23
```

```
* SQL>select last_day ('15-Aug-23') from dual;
```

```
LAST_DAY (
```

```
-----
```

```
o/p: 31-AUG-23
```

d) Next_day: - Based on the given date input and based on the weekday name input it provides the value.

example;

```
* SQL>select next_day (sysdate,'FRIDAY') from dual;
```

```
      NEXT_DAY (
```

```
      -----
```

```
o/p: 08-SEP-23
```

```
* SQL> select next_day (sysdate,'MONDAY') from dual;
```

```
      NEXT_DAY (
```

```
      -----
```

```
o/p: 11-SEP-23
```

e) current_date: - This function provides the current system date

example;

```
SQL>select current_date from dual;
```

```
      CURRENT_D
```

```
      -----
```

```
o/p: 07-SEP-23
```

Conversion Functions:

a) to_date: - This function converts the character representation of date into a standard Oracle date format

example;

```
* SQL>select to_date ('15-Aug-2023') from dual;
```

```
      TO_DATE ('
```

```
      -----
```

```
o/p: 15-AUG-23
```

```
* SQL>select to_date ('12/12/2023','DD/MM/YYYY') from dual;
```

```
      TO_DATE ('
```

```
      -----
```

```
o/p: 12-DEC-23
```

note: here we provide 12 and 12, it gets confused which is date, month and year so we need to explicitly specify which is date, month and year.

b) to_char: - This function converts the date into a character representation.

example;

```
* SQL>select to_char (sysdate, 'dd.mm.yyyy') from dual;
```

```
TO_CHAR (SY
```

```
-----
```

```
o/p: 07.09.2023
```

```
* SQL>select to_char (to_date ('10-SEP-2023'), 'mm,dd,yyyy') from dual;
```

```
TO_CHAR (TO
```

```
-----
```

```
o/p: 09.10.2023
```

note: here we give char value in single quotes it won't convert char to char.

c) to_number: - It converts char or String representation of integer to the standard integer value.

example;

```
SQL>select to_number ('10') from dual;
```

```
TO_NUMBER ('10')
```

```
-----
```

```
o/p:          10
```

Datatypes in SQL:

a) char(size): - This datatype is used to store char or alphanumeric value of fixed length, the data available here right padded with spaces

b) Varchar2(size): - This datatype is allowed to store variable length of character or alphanumeric data, the data available here is not right padded (rpad) with spaces.

c) number(size)/number (size, decimal): - This datatype is used to store integer value or floating value

example;

```
* Number(size);
```

```
number (10); here we can store data of 10 numbers.
```

```
* Number (size, decimal);
```

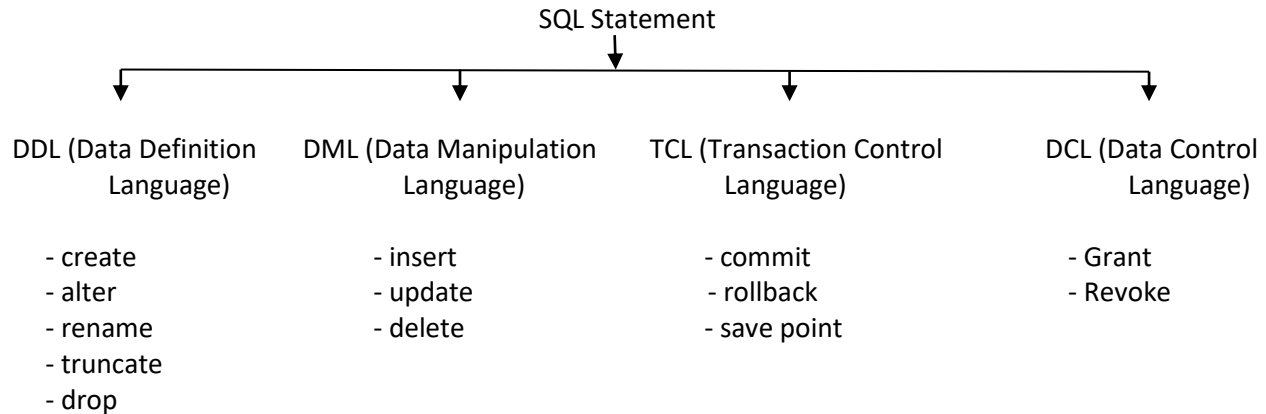
```
number (5,2); here we can store 5 data value in that 2 are decimal values
```

d) Date: - This datatype is used to store date value; the standard Oracle date format is DD/MON/YY

e) long: - This datatype is allowed to store alphanumeric data up to 2GB.

f) raw_long: - This datatype is used to store digitalized images and pictures.

SQL Statements: -



Difference between DDL and DML:

DDL	DML
*DDL stands for data definition language *DDL statements includes create, alter, rename truncate and drop. *All DDL statements are auto commit, it saves the transaction permanently *The DDL statements deals with manipulating the structure of the table	*DML stands for data manipulation language *DML statements includes insert, update, delete *DML statements are not auto commit in order to save the transaction we should use TCL statement 'commit' *DML statements deals with manipulating the data within the table

DDL Statements:

a) create: - By using create statement we can create the structure of the table.

*While creating the table we have to specify the name of the table, while providing the name

we have to follow the below rules:

- The name of the table never starts with integer
- The table name should not contain space for better readability we can use underscore.

example;

```
SQL>create table Product (PID number (2), Prodname varchar2(10), Price number (5.2),  
MFD date, Location varchar2(10));  
Table created.
```

How to display the structure of the table?

Ans: describe the table name

example;

```
SQL>desc Product;
Name                               Null?   Type
-----
PID                                NUMBER (2)
PRODNAME                           VARCHAR2(10)
PRICE                              NUMBER (5,2)
MFD                                DATE
LOCATION                             VARCHAR2(10)
```

note: Here Null? means by default tables gives null values

DML Statements: -

a) insert: - By using insert statement we can insert data into the table

case1: insert the data based on the sequence of the column

```
SQL> insert into product values (10,'DELL',125.75, sysdate-100,'Mysore');
1 row created.
```

note: here if we need to check the table we need to type 'select * from Product'.

```
SQL> select * from Product;
```

PID	PRODNAME	PRICE	MFD	LOCATION
10	DELL	125.75	30-MAY-23	Mysore

note: here * indicates the wild card datatype it provides all the column values which are available to the particular table.

case2: insert records based on the mapping of the column

```
SQL>insert into Product (PID, Prodname, Price, MFD, Location)
Values (11,'Lenovo',455.25, sysdate-150,'Bangalore');
1 row created.
```

```
SQL>select * from Product;
```

PID	PRODNAME	PRICE	MFD	LOCATION
10	DELL	125.75	30-MAY-23	Mysore
11	Lenovo	455.25	10-APR-23	Bangalore

case3: insert records into the table by substituting variable values.

```
SQL>insert into Product values (&Prodid, &Pname, &Amount, &MFD, &Place);
Enter value for prodid:12
Enter value for pname: 'Acer'
Enter value for amount:340.99
Enter value for mfd: sysdate-20
Enter value for place: 'Raichur'
old 1: insert into Product values (&Prodid, &Pname, &Amount, &MFD, &Place)
new 1: insert into Product values (12,'Acer',340.99, sysdate-20,'Raichur')
1 row created.
```

note: Here '&'(ampersand) indicates that it automatically creates the "Enter Value for"

* If we had to add data one more time

```
SQL>/
```

DDL Statements:

b) Alter: - By using alter table statement we can alter the structure of the table.

case1: Modify the size of the datatype

```
SQL>alter table Product Modify Location varchar2(20);
```

Table altered.

case2: How to add a new column for the existing table?

```
SQL>Alter table Product Add phoneNo number (10);
```

Table created.

case3: How to modify the datatype of the column?

- If the column is blank, we can modify source datatype to any destination datatype.
- If the column already contains data we can modify only with nearest datatype

```
SQL>Alter table Product modify phoneNo varchar2(10);
```

Table altered.

case4: How to rename the existing column name?

```
SQL>Alter table Product rename column phoneNo to MobileNo;
```

Table altered.

case5: How to drop the existing column?

```
SQL>Alter table Product drop column MobileNo;
```

Table altered.

Assignments:

1) Create the following table and insert min 10 records

a) Employee

b) Students

c) Insurance

d) Books

e) College

c) Rename: - By using rename statement we can rename the existing table

***How to create a new table based on the existing table structure and data?**

example;

```
SQL>create table product1 as select * from product;
```

Table created.

```
SQL> rename table product1 to productnew;
```

Table renamed.

d) Truncate: - The truncate statement removes all the records from the table permanently without affect into the structure of the table.

example;

```
SQL>truncate table productnew;
```

Table truncated.

e) Drop: - The drop statement removes all records from the table along with whole structure of the table permanently from database.

example;

```
SQL>drop table productnew;
```

Table dropped.

DML Statements:

DML stands for Data Manipulation Language, it includes insert, update, delete

a) update: - By using update statement we can update individual records or all the records.

case1: update individual records or a particular record

example;

```
SQL>update product set location='DALLAS' where pid=13;
```

1 row updated.

case2: How to update multiple columns data?

example;

```
SQL>update product set price=999.99, MFD=sysdate, location='Florida' where pid=13;
```

1row updated.

case3: How to update all records of a column?

example;

```
SQL>update product set location='Bangalore';
```

4 rows updated.

b) delete: - By using delete statement we can delete particular records or all the records from the table temporarily.

case1: How to delete a specific or particular record;

example;

```
SQL>delete from product where pid=13;
```

1 row deleted.

case2: How to delete all the records from table?

example;

```
SQL>delete from product;
```

3 rows deleted.

Difference b/w truncate and delete:

Truncate	Delete
*Truncate is a DDL statement *Truncate statement removes all the records from the table permanently without affecting to the structure of the table *Truncate statement is an auto commit it saves the transaction permanently *Truncate statement never accepts the where clause to filter the records *Truncate operation is very fast	*Delete is a DML statement *Delete statement deletes specific or all records from the table temporarily *Delete statement is not auto commit in order to save the transaction we should use TCL statements commit *Delete statement accepts the where clause to filter the records *Delete operation is slow compare to truncate

TCL Statements:

TCL stands for Transaction Control Language

*TCL statements can be applied only on DML statements

*TCL statements includes a) rollback
b) commit
c) Save point

a) Rollback: - Rollback statements can be applied only on the DML statements, its rollbacks to till last saved transaction

example;

SQL>rollback;

Rollback complete;

b) commit: - The commit statement can be applied only on DML statements, it saves the DML transaction permanently

example;

SQL>commit;

commit complete.

note: once we commit, we can't rollback

c) save point: - save point can be applicable only on DML statements by using save point we can save a particular DML transaction permanently and we can rollback a particular DML transaction based on the Save point.

example;

```
SQL>create table Employee (EmpNo number (2), EmpName varchar2(10),  
2 Jobname varchar2(10), salary number (4)),
```

Table created.

```
SQL>insert into Employee (10,'santosh','manager',9500);
```

1 row created.

```
SQL> select * from Employee;
```

EMPNO	EMPNAME	JOBNAME	SALARY
10	Santosh	Manager	9500

```
SQL>save point first;
```

Save point created.

in this way insert each record for each insertion of record create a separate save point.

note: A particular save point can be rollback by using below approach

```
<rollback to save point name>;
```

```
< > it defines the user defined statement;
```

DCL Statements:

DCL stands for Data Control language statements, by using DCL statements we can provide the permission and we can remove the permissions

DCL statements includes: -a) Grant
b) revoke

a) Grant: - By using grant statement the admin user can provide access permission to specific users

*Access permission includes create table or insert records or delete records etc.,

b) Revoke: - By using revoke statement the admin user can remove access permission to specific users

1) Find out the number of records available in the table?

-create a table student with 10 columns

-insert 5 records into student table

-update 3 records in student table

-insert 7 records into student table

- update 10 records in student table
- delete 8 records in student table
- update 2 records in student table
- insert 3 records in student table
- update 4 records in student table
- add a new column phoneNo in student table
- insert 3 records in student table
- delete 2 records from student table
- insert 10 records into student table
- update 5 records in student table
- insert 11 records in student table
- delete 4 records from student table
- insert 15 records into student table
- update 10 records in student table
- insert 21 records in student table
- delete 2 records from student table
- insert 11 records into student table
- update 20 records in student table
- insert 21 records into student table
- update 9 records from student table
- insert 19 records into student table
- delete 3 records in student table
- insert 11 records into student table
- update 10 records in student table
- insert 25 records into student table
- update 21 records in student table
- apply rollback statement and find out number of records available in a table

Ans: 7 records.

Constraints in SQL:

Constraints avoids the entry of invalid records into the table

* SQL supports in the following constraints

- Not null
- Unique
- Check
- Primary key
- Foreign key
- Default

a) Not Null Constraints: - The column which as applied by not null constraints that column never accept the blank/null value

note: The not null constraints can be applied in create table statement or in alter table statement

b) Unique Constraints: - The column which has applied by unique constraint that column never accepts duplicate values but it accepts null value

note: The unique constraints can be applied in create table statement or into alter table statement

c) Check Constraints: - Check constraints are also called business validation constraints based on the relational or logical operators we can specify the condition, based on the condition it validates the records

note: Check constraint can be applied in creatable statement or alter table statement

Case 1: Example for not null constraints;

```
SQL> create table Employee (EmpNo number (3), EmpName varchar2(10) constraints  
2    Empnotnull NOT NULL, JobName varchar2(10), salary number(5));
```

Table created.

```
SQL> insert into Employee values(10,'Santosh','Manager',7500);
```

1 row created.

```
SQL> select * from Employee;
```

EMPNO	EMPNAME	JOBNAME	SALARY
10	Santosh	Manager	7500

```
SQL> insert into Employee values (11,',','Manager',7500);
```

```
insert into Employee values (11,',','Manager',7500)
```

*

ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."EMPLOYEE"."EMPNAME")

Case2: Example for Unique constraints;

Continue from 1st case

```
SQL> alter table Employee add constraints Empunique UNIQUE(EMPNO);  
Table altered.
```

```
SQL> insert into Employee values(10,'Adams','Analyst',4500);  
insert into Employee values(10,'Adams','Analyst',4500)  
*
```

ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.EMPUNIQUE) violated

Case3: Example for Check constraints;

Continue from case 1&2

```
SQL> alter table Employee add constraints Empcheck CHECK (salary<=8000 and  
salary>=6000);
```

Table altered.

```
SQL> insert into Employee values(11,'Adams','Clerk',5999);  
  
insert into Employee values(11,'Adams','Clerk',5999)  
*
```

ERROR at line 1:
ORA-02290: check constraint (SYSTEM.EMPCHECK) violated

Note: How to display the constraints applied on the table

In Oracle there are 2 data dictionary tables are available

- a) User_constraints
- b) All_constraints

a) User_Constraints: -

```
SQL> select constraint_name, constraint_type, status from user_constraints where  
table_name='EMPLOYEE';
```

CONSTRAINT_NAME	C	STATUS
-----------------	---	--------

-----	- -----
EMPNOTNULL	C ENABLED
EMPUNIQUE	U ENABLED
EMPCHECK	C ENABLED

a) ALL_Constraints: -

SQL> select constraint_name, constraint_type, status from All_constraints where table_name='EMPLOYEE';

CONSTRAINT_NAME	C STATUS
-----	- -----
EMPNOTNULL	C ENABLED
EMPUNIQUE	U ENABLED
EMPCHECK	C ENABLED

How to know how many columns are available in Oracle SQL: -

SQL> desc user_constraints;

Name	Null?	Type
-----	-----	-----
OWNER		VARCHAR2(120)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(30)
CONSTRAINT_TYPE		VARCHAR2(1)
TABLE_NAME	NOT NULL	VARCHAR2(30)
SEARCH_CONDITION		LONG
R_OWNER		VARCHAR2(120)
R_CONSTRAINT_NAME		VARCHAR2(30)
DELETE_RULE		VARCHAR2(9)
STATUS		VARCHAR2(8)
DEFERRABLE		VARCHAR2(14)
DEFERRED		VARCHAR2(9)
VALIDATED		VARCHAR2(13)
GENERATED		VARCHAR2(14)
BAD		VARCHAR2(3)
RELY		VARCHAR2(4)
LAST_CHANGE		DATE
INDEX_OWNER		VARCHAR2(30)
INDEX_NAME		VARCHAR2(30)
INVALID		VARCHAR2(7)
VIEW_RELATED		VARCHAR2(14)

Note: If we didn't provide user defined constraints name while applying the constraints

```
SQL> create table Employee1 (EmpNo number(3) Unique, EmpName varchar2(10) NOT
NULL, JobName varchar2(10), salary number(5) CHECK(salary<=8000 and
salary>=6000));
```

Table created.

```
SQL> select constraint_name, constraint_type, status from user_constraints where
table_name='EMPLOYEE1';
```

CONSTRAINT_NAME	C	STATUS
-----	-	-----
SYS_C007004	C	ENABLED
SYS_C007005	C	ENABLED
SYS_C007006	U	ENABLED

Primary key Constraints:

Primary key is the combination of unique and not null constraint, the column which has applied by primary key that column never accept null records and duplicate records

Primary key has the following features:

- * Primary key constraint never accepts duplicate records
- * Primary key constraint never accepts null records
- * The table which has applied by primary key constraints that table is called as parent table
- * Only one primary key can be applied per table
- * If we can apply one primary key on more than one column of the same table that indicates composite primary key
- * The column which has applied by primary key constraint that column datatype should not be long
- * The main purpose of primary key is for record uniqueness

```
SQL> create table student (RollNo number (3), FirstName varchar2(10),
BranchName varchar2(15), DOJ date, Address varchar2(10));
```

Table created.

```
SQL> alter table student add constraint studentpk PRIMARY KEY (ROLLNO);
```

Table altered.

Validation of Primary Key:

```
SQL> insert into student values d(10,'Santosh','Computers', sysdate-200,'Mysore');
1 row created.
```

```
SQL> select * from student;
```

ROLLNO	FIRSTNAME	BRANCHNAME	DOJ	ADDRESS
-----	-----	-----	-----	-----
10	Santosh	Computers	21-FEB-23	Mysore

```
SQL> insert into student values (10,'Santosh','Computers', sysdate-200,'Mysore');
insert into student values (10,'Santosh','Computers', sysdate-200,'Mysore')
*
```

```
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.STUDENTPK) violated
```

```
SQL> insert into student values ('','Nandan', 'Computers', sysdate-100,'Hasan');
insert into student values ('','Nandan', 'Computers', sysdate-100,'Hasan')
*
```

```
ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."STUDENT"."ROLLNO")
```

Foreign key constraints:

The table which has applied by foreign key constraint that table is called as child table.
The records available in the child table must have reference in the parent table records, this is called as integrity constraint

There are two scenario we can violate the integrity constraint:

Scenario1: Deleting table directly from parent table that same record has reference in the child table

Scenario2: Inserting records directly into a child table without having the reference in the parent table

```
SQL> create table Library (LibID number (3), BookName varchar2(10),
AuthurName varchar2(10), Issued Date, RollNo number (3));
```

Table created.

```
SQL> Alter table Library add constraint libraryfk FOREIGN KEY(ROLLNO) references student
(ROLLNO);
```

Table altered.

```
SQL> insert into Library values(01,'Corejava','Santosh',sysdate-35,10);
```

1 row created.

```
SQL> insert into Library values(01,'Javascript','Kumar',sysdate-35,10);
```

1 row created.

```
SQL> select * from student;
```

ROLLNO	FIRSTNAME	BRANCHNAME	DOJ	ADDRESS
10	Santosh	Computers	21-FEB-23	Mysore

SQL> select * from Library;

LIBID	BOOKNAME	AUTHURNAME	ISSUED	ROLLNO
1	Corejava	Santosh	05-AUG-23	10
1	Javascript	Kumar	05-AUG-23	10

Integrity constraint violation:

Case1: Deleting records directly from the parent table that same record has the reference in the child table

SQL> delete from student where ROLLNO=10;

delete from student where ROLLNO=10

*

ERROR at line 1:

ORA-02292: integrity constraint (SYSTEM.LIBRARYFK) violated - child record found

Case2: Inserting records directly into a child table without having the reference in the parent table

SQL> insert into library values (3,'SQL','Rohith', sysdate-10,15);

insert into library values (3,'SQL','Rohith', sysdate-10,15)

*

ERROR at line 1:

ORA-02291: integrity constraint (SYSTEM.LIBRARYFK) violated - parent key not found

Assignments

- 1) Student, Library, sports, computer department
 - 2) Department, Employee, Insurance, Transport
 - 3) Customers, Product, Transaction
 - 4) Product, Sales, Purchase, Orders
- Write a relation using primary and foreign key for the above conditions

Default Constraint: -

By using Default constraint keyword we can insert default value while inserting records into the table

Example;

```
SQL> create table employee3(empno number(3),empname varchar2(10),job varchar2(10),salary
    number(5) default 5000);
```

Table created.

```
SQL> insert into employee3 values(10,'Ranav','Manager',6000);
```

1 row created.

```
SQL> insert into employee3 values(11,'Rakshi','Manager',default);
```

1 row created.

```
SQL> select * from employee3;
```

EMPNO	EMPNAME	JOB	SALARY
-----	-----	-----	-----
10	Ranav	Manager	6000
11	Rakshi	Manager	5000

How to disable the constraints?

From user_constraint table list out all available constraint for a specific table

```
SQL> select constraint_name, status from user_constraints where table_name='EMPLOYEE';
```

CONSTRAINT_NAME	STATUS
-----	-----
EMPNOTNULL	ENABLED
EMPUNIQUE	ENABLED
EMPCHECK	ENABLED

```
SQL> alter table employee disable constraint empnotnull;
```

Table altered.

```
SQL> select constraint_name, status from user_constraints where table_name='EMPLOYEE';
```

CONSTRAINT_NAME	STATUS
-----	-----
EMPNOTNULL	DISABLED
EMPUNIQUE	ENABLED
EMPCHECK	ENABLED

How to Enable the constraints?

While enabling the constraint the table should not have any invalid records in a table, if table already contains invalid records in this case we can't enable the constraint, first we have to identify the invalid record and delete the same record then only we can enable the constraint

```
SQL> insert into employee values(11, 'Manager', 7000);
```

1 row created.

```
SQL> select * from employee;
```

EMPNO	EMPNAME	JOBNAME	SALARY
10	Santosh	Manager	7500
11	Manager		7000

```
SQL> alter table employee enable constraint empnotnull;
```

```
alter table employee enable constraint empnotnull
*
```

ERROR at line 1:

ORA-02293: cannot validate (SYSTEM.EMPNOTNULL) - check constraint violated

```
SQL> delete from employee where empno=11;
```

1 row deleted.

```
SQL> alter table employee enable constraint empnotnull;
```

Table altered.

```
SQL> select constraint_name, status from user_constraints where table_name='EMPLOYEE';
```

CONSTRAINT_NAME	STATUS
EMPNOTNULL	ENABLED
EMPUNIQUE	ENABLED
EMPCHECK	ENABLED

a) On DELETE CASCADE: -

ON DELETE CASCADE apply only on foreign key constraint, if you delete any records from the parent table the corresponding reference records from the child table deletes automatically

Step1: Display all records from student table

```
SQL> select * from student;
```

ROLLNO	FIRSTNAME	BRANCHNAME	DOJ	ADDRESS
-----	-----	-----	-----	-----
10	Santosh	Computers	21-FEB-23	Mysore

Step2: Display all records from library table

SQL>select * from library;

LIBLID	BOOKNAME	AUTHORNAME	ISSUED	ROLLNO
-----	-----	-----	-----	-----
1	Corejava	Williams	15-AUG-23	10
2	Javascript	BalaGuru	28-AUG-23	10

Step3: Drop constraint foreign key from table library

SQL>alter table library drop constraint libraryfk;

Table altered.

Step4: add on delete cascade to child table

SQL> alter table library add constraint libraryfk FOREIGN KEY(ROLLNO) references student (ROLLNO) on delete cascade;

Table altered.

Step5: Delete all records from parent table

SQL> delete from student where ROLLNO=11;

1 row deleted.

Step6: display all records from library

SQL> select * from library;

No rows selected.

b) ON DELETE SET NULL: -

On delete set null can be applied only on foreign key constraint, if we delete any records from the parent table the corresponding reference record from child table set to null

Step1: Display all records from student table

SQL> select * from student;

ROLLNO	FIRSTNAME	BRANCHNAME	DOJ	ADDRESS
10	Santosh	Computers	21-FEB-23	Mysore

Step2: Display all records from library table

SQL>select * from library;

LIBLID	BOOKNAME	AUTHORNAME	ISSUED	ROLLNO
1	Corejava	Williams	15-AUG-23	10
2	Javascript	BalaGuru	28-AUG-23	10

Step3: Drop constraint foreign key from table library

SQL>alter table library drop constraint libraryfk;

Table altered.

Step4: add on delete set null to child table

SQL> alter table library add constraint libraryfk FOREIGN KEY(ROLLNO) references
student (ROLLNO) on delete set null;

Table altered.

Step5: Delete all records from parent table

SQL> delete from student where ROLLNO=11;
1 row deleted.

Step6: display all records from library

SQL> select * from library;

LIBLID	BOOKNAME	AUTHORNAME	ISSUED	ROLLNO
1	Corejava	Williams	15-AUG-23	null
2	Javascript	BalaGuru	28-AUG-23	null

Normalization in SQL:

Normalization is used to organize the data in such manner that data redundancy will never occur in the database and data should not be in inconsistent manner.

There are 3 forms of Normalization:

First Normal Form (1NF): If the table has to satisfy 1NF, each column in a table should have atomic value, column should not contain repeating group of values.

Friends:

FID	Friends name	Favorite Actors
1	Ajay	Puneeth, darshan, Sudeep
2	Arjun	Salman, Amir, Akshay
3	Ajith	Yash, Ganesh, Rakshith

The above table is not in 1NF. currently it does not have atomic value in favorite actor column. In order to apply atomic value for the existing table, we should design the table as follows.

Fid	Friend Name	Favorite Actor1	Favorite Actor2	Favorite Actor3
1	Ajay	Puneeth	Darshan	Sudeep
2	Arjun	Salman	Amir	Akshay
3	Ajith	Yash	Ganesh	Rakshith

The above table displays the repeating value in each column. so in order to apply the 1NF we should split the table and apply the one to one relationship between the two tables as follows

Fid	Friend name
1	Ajay
2	Arjun
3	Ajith

Fid	Friend Name
1	Puneeth
1	Darshan
1	Sudeep
2	Salman
2	Amir
2	Akshay
3	Yash
3	Ganesh
3	Rakshith

Second Normal Form (2NF): The table must be in the 1NF, every non key column value must entirely depend on primary key (Functional dependency).

There should be a functional dependency for every 2NF.

Gadget:

Gadget	Supplier	Cost	Address
Headphone	Sony	15\$	Bangalore
Pendrive	HP	7\$	Chennai
Headphone	Samsung	20\$	Delhi

- * In the above table, if you know the gadget can we estimate the cost? No
- * If we know the supplier, can we estimate cost? No
- * The cost column values are entirely depending on gadget and supplier value.
- * If we know the gadget name, can we know the address? No
- * If we know the supplier, can we know the address? Yes
- * In the above description, the address column contains non key value, this value entirely depends upon supplier column. so, in order to apply the 2NF on above table, we should redesign the table as follows.

Gadget	Supplier	Cost
Headphone	Sony	15\$
Pendrive	HP	7\$
Headphone	Samsung	20\$

Supplier	Address
Sony	Bangalore
HP	Chennai
Samsung	Delhi

Third Normal Form (3NF): The table must be in 2NF, there should not be any transitive dependency.

Transitive dependency: A non-key column depends on another non key column that indicates the Transitive dependency.

Album:

Album Name	Singer	# of Tracks	Country
Riverside	Scott	4	France
Jaipari	Alan	2	India
Terachehara	Smith	8	Japan
Downstream	Alan	6	India

- *If we know the album name, can we get the country name? No
- *If we know the singer's name, can we get the country name? Yes
- *In the above table based on the singer's name we can identify the country name. currently the singer name column contains non key value. The country column contains non key value so it provides the transitive dependency.
- *In order to eliminate transitive dependency that table should be redesigned as follows.

Album Name	Singer	# of Tracks
Riverside	Scott	4
Jaipari	Alan	2
Terachehara	Smith	8
Downstream	Alan	6

Singer	Country
Scott	France
Alan	India
Smith	Japan
Alan	India

Part2:

Configure Emp and Dept tables: -

How to get EMP and DBT tables?

Ans: go to Oracle installed path that is in C drive there navigate the path
I.e., c:\oracle\app\oracle\product\11.2.0\server\rdbms\admin

Search for scott.sql file and copy the scott.sql file go to any one of the drive paste this particular folder

How to install this SQL

SQL> @ G:\EXAMPLE\scott.sql;

View records from the table: -

How to set the line and page size to display the data?

Ans: SQL> select * from EMP;
SQL> select * from DBT;

SQL> set linesize 120;
SQL> set pagesize 200;

How to display records from the table?

Case1: Display selected columns from the table

SQL> select ename,job,sal from emp;

ENAME	JOB	SAL
-------	-----	-----

-----	-----	-----
SMITH	CLERK	800
ALLEN	SALESMAN	1600
WARD	SALESMAN	1250
JONES	MANAGER	2975
MARTIN	SALESMAN	1250
BLAKE	MANAGER	2850
CLARK	MANAGER	2450
SCOTT	ANALYST	3000
KING	PRESIDENT	5000
TURNER	SALESMAN	1500
ADAMS	CLERK	1100
JAMES	CLERK	950
FORD	ANALYST	3000
MILLER	CLERK	1300

14 rows selected.

Case2: Display selected columns and selected rows

SQL> select ename,job,sal from emp where job='salesman';

ENAME	JOB	SAL
-----	-----	-----
ALLEN	SALESMAN	1600
WARD	SALESMAN	1250
MARTIN	SALESMAN	1250
TURNER	SALESMAN	1500

Case3: Display all columns and selected rows

SQL> select * from emp where job='ANALYST';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Case4: Display all column and all rows from the table

SQL> select * from emp;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTN
-----	-----	-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10

7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

14 rows selected.

Operators available in SQL: -

1) Arithmetic operators:

+ ---> Addition
 - ----> Subtraction
 * ---> Multiplication
 / ----> Division
 Mod ----> Modulus

2) Relational operators:

> ---> Greater than
 < ---> Less than
 >= ---> Greater than equal to
 <= ---> Less than equal to
 = ---> Equal to
 != ---> Not equal to

3) Concatenation operator:

||---> Concatenation operator

Note: || pipe symbol -> shift + backspace

4) Logical operator:

And ---> and operator

A	B	A and B
0	0	0
1	0	0
0	1	0
1	1	1

Note: If both condition is true it will true otherwise false;

Or ---> or operator

A	B	A and B
0	0	0
1	0	1
0	1	1
1	1	1

Note: if any one condition is true then it is true otherwise false;

Not ---> not operator

A	Not A
0	1
1	0

Note: It is an inverse operation

Apply operators on table data: -

Case1: Apply Arithmetic operator

Desc: WAQ to display per Anum salary of each employee?

Ans: SQL> select ename, sal*12 "SALARY" from emp;

ENAME	SALARY
-----	-----
SMITH	9600
ALLEN	19200
WARD	15000
JONES	35700
MARTIN	15000
BLAKE	34200
CLARK	29400
SCOTT	36000
KING	60000
TURNER	18000
ADAMS	13200
JAMES	11400
FORD	36000
MILLER	15600

14 rows selected.

Case2: Apply relational operators

Desc: WAQ to display all the employee details their salary is greater than 3000?

SQL> select * from emp where sal>3000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7839	KING	PRESIDENT		17-NOV-81	5000	10	

Case3: Apply concatenation Operators

Desc: WAQ to create a sentence from column data

SQL> select 'The Department '||initcap(dname)||' exists in the city '||initcap(loc) from dept;

'THEDEPARTMENT' || INITCAP(DNAME) || 'EXISTSINTHECITY' || INITCAP(LO

The Department Accounting exists in the city New York

The Department Research exists in the city Dallas
 The Department Sales exists in the city Chicago
 The Department Operations exists in the city Boston

Case4: Apply logical operators

Desc: WAQ to display all employees details their salary in between 1000 and 2000?

SQL> select * from emp where sal>=1000 and sal<=2000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

6 rows selected.

Distinct clause: -

The distinct clause provides one occurrence of the record from the duplicate set of records and it also describes as it provides distinct records

SQL> select distinct deptno from emp;

DEPTNO
30
20
10

In case we use unique column to display then distinct clause never works distinct clause works only on duplicate records I e.,

SQL> select distinct deptno,ename from emp;

DEPTNO	ENAME
20	JONES
30	WARD
20	SCOTT
10	KING
30	JAMES
30	ALLEN
30	MARTIN
30	BLAKE
20	FORD

20	SMITH
20	ADAMS
10	MILLER
10	CLARK
30	TURNER

14 rows selected.

Pattern Matching (Like or Not Like): -

By using pattern matching we can achieve partial matching of the records, it can be achieved by using following inbuilt characters

% ---> It matches more than one character

_ (Underscore)---> It matches only one character at a time

Like Operator: -

Case1: Starts With

Desc: WAQ to display Employee details in which job starts with 'SALES'

SQL> select * from emp where job like 'SALES%';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30

Case2: Ends with

Desc: WAQ to display employee details in which job ends with 'MAN'

SQL> select * from emp where job like '%MAN';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30

Case3: Contains

Desc: WAQ to display employee details in which their hiredate is DEC

SQL> select * from emp where hiredate like '%DEC%';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-------	-------	-----	-----	----------	-----	------	--------

7369	SMITH	CLERK	7902	17-DEC-80	800	20
7900	JAMES	CLERK	7698	03-DEC-81	950	30
7902	FORD	ANALYST	7566	03-DEC-81	3000	20

Case4: Underscore (It matches exactly one character at a time)

Desc: WAQ to display employee details in which employee name contains 4 characters

SQL> select * from emp where ename like '____';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Desc: WAQ to display employee details in which employee name contains 4 characters

And ends with RD

SQL> select * from emp where ename like '____RD';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Not Like/ Inverse Operator: -

Desc: WAQ to display all employee details their hiredate is not in September (SEP) month

SQL> select * from emp where hiredate not like '%SEP%';

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

12 rows selected.

In case if we need more than one month use and operator

```
SQL> select * from emp where hiredate not like '%SEP%' and hiredate not like '%DEC%';
```

Note: we can use not operator before the column name or after

```
SQL> select * from emp where not hiredate like '%SEP%';
```

Display records using in or not in operator (In or Not In): -

In operator:

The IN operator is used to filter the list of values, each value available in the list has applied 'OR' operator here

Desc: WAQ to display employee name their ename KING, SCOTT, FORD and RAVI

```
SQL> select * from emp where ename IN('KING','SCOTT','FORD','RAVI');
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Desc: WAQ to display employee name their empno 7902,7900,8080,8081

```
SQL> select * from emp where empno IN (7902,7900,8080,8081);
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Note: Here it filters the list of values using 'OR' operator then it provides only available and ignores the not available records

Not IN (Inverse Operators):

It is an inverse operator to IN operator

Desc: WAQ to display employee details their job other than salesman and manger

```
SQL> select * from emp where job not in('SALESMAN','MANAGER');
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30

7902	FORD	ANALYST	7566	03-DEC-81	3000	20
7934	MILLER	CLERK	7782	23-JAN-82	1300	10

7 rows selected.

Or

SQL> select * from emp where not job in('SALESMAN','MANAGER');

Between Operator (Between or Not Between): -

Between operator:

Between operator is used to filter the range of values

Desc: WAQ to display employee details their salary in between 3000 to 5000

SQL> select * from emp where sal between 3000 and 5000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Rules for Between operator:

- In Between operator the lower limit should be lesser than the upper limit
- In between operator and is the inbuilt keyword

Not Between (Inverse Operator):

It is an inverse operator to between operator

Desc: WAQ to display employee details salary not in the range of 1000 to 3000

SQL> select * from emp where sal not between 1000 and 3000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7900	JAMES	CLERK	7698	03-DEC-81	950		30

Or

SQL> select * from emp where not sal between 1000 and 3000;

Order by clause (Sorting the records): -

The Order by clause is used to sort the records either ascending order or descending order, by default it sorts the records in ascending order

Desc: WAQ to display all employee details in which employee name sorted in ascending order

SQL> select * from emp order by ename ASC;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30

14 rows selected.

Desc: WAQ to display all employee details in which employee name sorted in descending order

SQL> select * from emp order by ename DESC;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7839	KING	PRESIDENT		17-NOV-81	5000		10
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20

14 rows selected.

Desc: By default, it sorts the records in ascending order

SQL> select * from emp order by ename;

Case4: Apply the sort operation on more than one column

SQL> select * from emp order by ename,job ASC;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7698	BLAKE	MANAGER	7839	01-MAY-81		2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30

14 rows selected.

Aggregate Functions: -

This function provides the specified value from the specified column

Min: This function provides lowest or minimum value from the specified column

SQL> select min(sal) from emp;

MIN(SAL)

800

Max: This function provides Highest or maximum value from the specified column

SQL> select max(sal) from emp;

MAX(SAL)

5000

AVG: This function provides Average value from the specified column

SQL> select avg(sal) from emp;

AVG(SAL)

```
-----  
2073.21429
```

Sum: This function provides Sum of all records available in a specified column

```
SQL> select sum(sal) from emp;  
SUM(SAL)
```

```
-----  
29025
```

Count: This function provides number of records available in a specified column

```
SQL> select count(sal) from emp;
```

```
COUNT(SAL)
```

```
-----  
14
```

```
SQL> select count(comm) from emp;
```

```
COUNT(COMM)
```

```
-----  
4
```

Count (*): This function provides number of records available in a table

```
SQL> select count (*) from emp;
```

```
COUNT (*)
```

```
-----  
14
```

Note: here Count (*) gives the highest column number/ highest record available in the table

Set Operations: -

Set operations are used to combine the records from same table or different table

The set operation includes:

- Union
- Union All
- Intersect
- Minus

All set operations work based on the following rules:

- The number of columns used in the first query must be equal to the number of columns used in the second query
- Datatype of each column specified in the first query must be match with datatype of each column specified in the second query

- a) **Union:** Union operation is used to combine the records from same table or different tables, the union operation returns distinct records

```
SQL> select deptno from emp
      UNION
      Select deptno from dept;
```

```
DEPTNO
-----
10
20
30
40
```

Note: The output of union operation is always sorted in ascending order

General example;

```
A= {1.2.3}
B= {2.3.4.5}
A Union B= {1,2,3,4,5}
```

b) Union All: Union all operation is used to combine the records from same table or multiple tables, the union all operation provides duplicate records

Example;

```
A= {1.2.3}
B= {2.3.4.5}
A Union All B= {1,2,3,2,3,4,5}
```

```
SQL> select deptno from emp
      Union All
      Select deptno from dept;
```

```
DEPTNO
-----
20
30
30
20
30
30
10
20
10
30
20
```

Note: The output of the Union All operation is not in sorted order

c) Intersect: The intersect operation returns common records between two given queries, it returns distinct records and the output of intersect operation is always sorted in ascending order

Example;

A= {1.2.3}
B= {2.3.4.5}
A Intersect B= {2,3}

```
SQL> select deptno from emp
      INTERSECT
      select deptno from dept;
```

```
DEPTNO
-----
10
20
30
```

d) Minus: The minus operation provides records from first query the same records which are not available in the second query

Example;

A= {1.2.3}
B= {2.3.4.5}
A Minus B= {1}
B Minus A= {4,5}

```
SQL> select deptno from emp
      Minus
      select deptno from dept;
```

```
DEPTNO
-----
No rows selected
```

```
SQL> select deptno from dept
      Minus
      select deptno from emp;
```

```
DEPTNO
-----
40
```

Validation of set operation rules:

Case1: Validation of rule1

```
SQL> select empno, ename from emp
      union
      select deptno from dept;
```

```
select empno, ename from emp
*
```

ERROR at line 1:
ORA-01789: query block has incorrect number of result columns

Case2: Validation of rule2

```
SQL> select ename from emp
      union
      select deptno from dept;
```

```
select ename from emp
*
```

ERROR at line 1:
ORA-01790: expression must have same datatype as corresponding expression

Group By Clause: -

Group by clause is used to group the records based on the aggregate functions

Group by Clause Rules;

- The column name specified in the select statement the same column name must specified in the group by clause
- After satisfying the first rule, the column name specified in the group by clause the same column name need to be specified in the select statement

Case1: Query as per the rule1

WAQ to display jobwise employee count

```
SQL> select job, count(ename) from emp group by job;
```

JOB	COUNT(ENAME)
-----	-----
CLERK	4
SALESMAN	4
PRESIDENT	1
MANAGER	3
ANALYST	2

Note: The ename specified by count is not a column it is a reference to count/ aggregate function

Syntax:

```
SQL> select col1, col2, ...colN aggregate function1, aggregate function2, ... aggregate functionN from
      <table name> where<condition> group by col1, col2, ...colN;
```

If I need to filter the records use where <condition> be i.e.

```
SQL> select job, count(ename) from emp where job in ('Manager', 'Analyst') group by job;
```

JOB	COUNT(ENAME)
-----	-----

MANAGER	3
ANALYST	2

Note: we can't use where condition after the group by clause

Case2: Query as per the second rule

WAQ to display the employee count based on job and deptno

SQL> select job, count(ename)from emp group by job, deptno;

JOB	COUNT(ENAME)
-----	-----
MANAGER	1
PRESIDENT	1
CLERK	1
SALESMAN	4
ANALYST	2
MANAGER	1
MANAGER	1
CLERK	1
CLERK	2

9 rows selected.

SQL> select job, deptno, count(ename)from emp group by job, deptno;

JOB	DEPTNO	COUNT(ENAME)
-----	-----	-----
MANAGER	20	1
PRESIDENT	10	1
CLERK	10	1
SALESMAN	30	4
ANALYST	20	2
MANAGER	30	1
MANAGER	10	1
CLERK	30	1
CLERK	20	2

9 rows selected.

Special cases:

Case1: Can we use multiple aggregate functions in a single select statement

Ans: Yes

SQL> select max(sal), min(sal), sum(sal) from emp;

MAX(SAL)	MIN(SAL)	SUM(SAL)
-----	-----	-----
5000	800	29025

Case2: What is the output of the below query

```
SQL> select ename, max(sal) from emp;
```

```
Select ename, max(sal) from emp
```

```
*
```

ERROR at line1:

In select statement along with column name if you specify aggregate function in this case we must specify the group by clause.

HAVING Clause:

Having clause is used to filter the grouped record

Difference Between Where clause and Having Clause

Where Clause	Having Clause
* Where clause is used to filter the individual Records	* Having clause is used to filter the grouped record
* In where clause we should not use aggregate Function to filter the record	* In having clause we must use aggregate function to filter the record
* Where clause can be used before the group By clause	* Having clause can be used after the group by clause

Rules applicable for the Having Clause;

- In Having Clause, we must use aggregate functions to filter the grouped records
- The aggregate functions which are used in the select statement the same or different aggregate functions mite be used in the having clause to filter the grouped records

Syntax:

```
SQL> select col1, col2,...colN, aggregate function1, aggregate function2,...aggregate function N
from<Table name> where<condition>
Group by col1, col2,..colN
Having <aggregate function with condition>
Order by <col1, col2,...>
```

WAQ to display jobwise min and max salary

```
SQL> select job, min(sal), max(sal) from emp group by job;
```

JOB	MIN(SAL)	MAX(SAL)
-----	-----	-----
CLERK	800	1300
SALESMAN	1250	1600
PRESIDENT	5000	5000

MANAGER	2450	2975
ANALYST	3000	3000

Or

SQL> select job, min(sal), max(sal) from emp group by job having(min(sal)>=2000);

JOB	MIN(SAL)	MAX(SAL)
-----	-----	-----
PRESIDENT	5000	5000
MANAGER	2450	2975
ANALYST	3000	3000

Note: Here we using Having clause for the condition to display the particular records based on condition

WAQ to display/ satisfy W G H O

SQL> select job, min(sal), max(sal) from emp where job in('MANAGER','ANALYST','PRESIDENT')
group by job having (min(sal)>=2000)
order by job ASC;

JOB	MIN(SAL)	MAX(SAL)
-----	-----	-----
ANALYST	3000	3000
MANAGER	2450	2975
PRESIDENT	5000	5000

Display the records based on the Null Value: -

IS NULL/ IS NOT NULL:

IS NULL:

WAQ to display the employee's name who is not reporting to any person

SQL> select ename from emp where MGR is null;

ENAME

KING

WAQ to display the employee details who are not getting commission

SQL> select * from emp where comm is null;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	7902	17-DEC-80	800		20

7566	JONES	MANAGER	7839	02-APR-81	2975	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850	30
7782	CLARK	MANAGER	7839	09-JUN-81	2450	10
7788	SCOTT	ANALYST	7566	19-APR-87	3000	20
7839	KING	PRESIDENT		17-NOV-81	5000	10
7876	ADAMS	CLERK	7788	23-MAY-87	1100	20
7900	JAMES	CLERK	7698	03-DEC-81	950	30
7902	FORD	ANALYST	7566	03-DEC-81	3000	20
7934	MILLER	CLERK	7782	23-JAN-82	1300	10

10 rows selected.

IS NOT NULL:

WAQ to display the employee details who are reporting to corresponding Manager

SQL> select * from emp where MGR is NOT NULL;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

13 rows selected.

WAQ to display employee details who are getting commission

SQL> select * from emp where COMM is NOT NULL;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30

Sub Queries In SQL: -

Sub queries are also called as nested queries, in sub query the inner query executes first based on the result of the inner query the outer query executes

Sub query can be applied on the same table or multiple table

WAQ to display the second max salary

```
SQL> select max(sal) from emp;
```

```
MAX(SAL)
-----
      5000
```

```
SQL> select max(sal) from emp where sal<5000;
```

```
MAX(SAL)
-----
      3000
```

Or

```
SQL> select max(sal) from emp where sal< (select max(sal) from emp);
```

```
MAX(SAL)
-----
      3000
```

Note: In where clause we should not use aggregate function to filter the records, so don't write query in this way

```
SQL> select max(sal) from emp where sal< max(sal);
```

WAQ to display the second min salary

```
SQL> select min(sal) from emp;
```

```
MIN(SAL)
-----
       800
```

```
SQL> select min(sal) from emp where sal>800;
```

```
MIN(SAL)
-----
       950
```

Or

```
SQL> select min(sal) from emp where sal > (select min(sal) from emp);
```

```
MIN(SAL)
-----
      950
```

Note: In where clause we should not use aggregate function to filter the records, so don't write query in this way

```
SQL> select min(sal) from emp where sal < min(sal);
```

WAQ to display third max salary

```
SQL> select max(sal) from emp where sal < (select max(sal) from emp where sal < (select max(sal) from emp));
```

```
MAX(SAL)
-----
      2975
```

WAQ to display the fifth max salary

```
SQL> select max(sal) from emp where sal < (select max(sal) from emp where sal <
(select max(sal) from emp where sal <
(select max(sal) from emp where sal <
(select max(sal) from emp))));
```

```
MAX(SAL)
-----
      2450
```

WAQ to display the employee's name who is getting highest salary

```
SQL> select ename from emp where sal = (select max(sal) from emp);
```

```
ENAME
-----
KING
```

Sub query on multiple table: -

WAQ to display dname in which employees are working

```
SQL> select dname from dept where deptno in (select deptno from emp);
```

```
DNAME
-----
ACCOUNTING
RESEARCH
SALES
```


WAQ to display dname in which employees are not working

```
SQL> select dname from dept where deptno NOT in (select deptno from emp);
```

```
DNAME
```

```
-----
```

```
OPERATIONS
```

Rules for Sub queries in SQL:

- In Sub query the column used in the outer query where clause that column matching records must be provided by the inner query

WAQ to display the deptnames in which employees are getting commission

```
SQL> select dname from dept where deptno in (select deptno from emp where comm is not null);
```

```
DNAME
```

```
-----
```

```
SALES
```

WQA to display the deptnames in which president is working

```
SQL> select dname from dept where deptno in (select deptno from emp where job = 'PRESIDENT');
```

```
DNAME
```

```
-----
```

```
ACCOUNTING
```

W G H O Rule:

W ---> Where clause

G ---> Group by Clause

H ---> Having by Clause

O ---> Order by Clause

Wherever we want to check query that whether it satisfy the rule or not i.e.,

W G H O

W G

W G H

G H

G H O

W O

G O

INLINE Query:

While writing the query in place of table name if we are writing a query that indicates a concept of Inline query

```
SQL> select * from dept;
```

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Instead

SQL> select * from (select * from dept);

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

RowNum: - Rownum is a sequentially generated number for each record available in a table, the rownum always starts with 1, the insert or delete operation can change the rownum

Note:

How to display the RowNum from a table?

SQL> select deptno, dname, loc, rownum from dept;

DEPTNO	DNAME	LOC	ROWNUM
-----	-----	-----	-----
10	ACCOUNTING	NEW YORK	1
20	RESEARCH	DALLAS	2
30	SALES	CHICAGO	3
40	OPERATIONS	BOSTON	4

Case1: Can we apply rownum in IN operator?

Ans: Yes

SQL> select * from dept where rownum in (1,2,3);

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO

if we skip row 1 then

SQL> select * from dept where rownum in (2,3,4);
no rows selected

note: If we skip row1 it never works because RowNum always starts with 1, if the reference is missing it throws the error

case2: RowNum in between operator

```
SQL> select * from dept where rownum between 1 and 3;
```

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO

If we skip row1

```
SQL> select * from dept where rownum between 2 and 4;
```

no rows selected

note: because the reference is missing and it throw the error

case3: Apply relational operator with RowNum

```
SQL> select * from dept where rownum<=3;
```

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO

If we skip row1

```
SQL> select * from dept where rownum>=2;
```

no rows selected

note: because the reference rownum1 is missing whatever we use ultimately it should contain reference rownum 1

WAQ to find out first max salary without using aggregate function

```
SQL> select sal from (select sal from emp order by sal desc) where rownum=1;
```

SAL

5000

WAQ to find the 2nd max salary without using aggregate functions

STEP1: List out all available salary records from the emp table

```
SQL> select sal from (select sal from emp);
```

Step2: Make list of salary column in desc order

```
SQL>select rownum, sal from (select sal from emp order by sal desc);
```

Step3: Make a list of salary based on kk in desc order

```
SQL> select kk, sal from (select rownum as kk, sal from (select sal from emp order by sal desc));
```

KK	SAL
-----	-----
1	5000
2	3000
3	3000
4	2975
5	2850
6	2450
7	1600
8	1500
9	1300
10	1250
11	1250
12	1100
13	950
14	800

14 rows selected.

Step4: Remove the duplicate salary records

```
SQL> select kk, sal from (select rownum as kk, sal from (select distinct sal from emp order by sal desc));
```

KK	SAL
-----	-----
1	5000
2	3000
3	2975
4	2850
5	2450
6	1600
7	1500
8	1300
9	1250

10	1100
11	950
12	800

12 rows selected.

Step5: Find the 2nd max salary from table

```
SQL> select sal from (select kk, sal from (select rownum as kk, sal from (select distinct sal from emp
order by sal desc)) where kk=2);
```

```
SAL
-----
3000
```

Note: here I am using stepwise for better understanding purpose the step5 is the main step to find the 2nd max salary without using aggregate functions

WAQ to find the nth max salary without using aggregate functions

```
SQL> select sal from (select kk, sal from (select rownum as kk, sal from (select distinct sal from emp
order by sal desc)) where kk=&n);
```

Enter value for n: 2

```
SAL
-----
3000
```

Note:

Nth Value: which is the input we provide to the nth value the respective max value for that nth value will provide

WAQ to display 1st half of the records without using aggregate functions

```
SQL> select * from emp where rownum<= (select count(*)/2 from emp);
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
-----	-----	-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10

7 rows selected.

WAQ to display 2nd half of the records without using aggregate functions with sorting existing columns

```
SQL> select * from (select * from emp order by empno desc) where rownum<= (select count (*)/2
from emp) order by empno ASC;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

7 rows selected.

WAQ to find the 2nd half of the records without using aggregate functions and without sorting the existing columns

Step1: select new column to sort the order i.e column kk

```
SQL> select kk, empno, ename, job, mgr, hiredate, sal, comm, deptno from(select rownum as
kk, empno, ename, job, mgr, hiredate, sal, comm, deptno from emp) order by kk DESC;
```

KK	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
14	7934	MILLER	CLERK	7782	23-JAN-82	1300		10
13	7902	FORD	ANALYST	7566	03-DEC-81	3000		20
12	7900	JAMES	CLERK	7698	03-DEC-81	950		30
11	7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
10	7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
9	7839	KING	PRESIDENT		17-NOV-81	5000		10
8	7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7	7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
6	7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
5	7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
4	7566	JONES	MANAGER	7839	02-APR-81	2975		20
3	7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
2	7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
1	7369	SMITH	CLERK	7902	17-DEC-80	800		20

14 rows selected.

Step2: find the 2nd half

```
SQL> select empno, ename, job, mgr, hiredate, sal, comm, deptno from(select kk, empno, ename,
job, mgr, hiredate, sal, comm, deptno from(select rownum as kk, empno, ename, job, mgr,
```

hiredate, sal, comm, deptno from emp) order by kk DESC)where kk>(select count(kk)/2 from emp) order by kk ASC;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

7 rows selected.

WAQ to display the middle of the records without using aggregate function and existing column

SQL> select empno, ename, job, mgr, hiredate, sal, comm, deptno from (select kk, empno, ename, job, mgr, hiredate, sal, comm, deptno from(select rownum as kk, empno, ename, job, mgr, hiredate, sal, comm, deptno from emp)) where kk between 5 and 8;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20

ROWID: -

Rowid is a fixed 16-to-18-digit character of alphanumeric value assign to each row available in a table, Rowid is always unique it never changes

WAQ to display the Rowid

Step1: Create a table

SQL> create table student2(ID number (2), Sname varchar2(10), location varchar2(10));

Table created.

Step2: Insert duplicate records into the table and display the records

SQL> select * from student2;

ID	SNAME	LOCATION
10	Santosh	Kalburgi
10	Santosh	Kalburgi
10	Santosh	Kalburgi

10	Santosh	Kalburgi
11	Pallavi	Mysore
11	Pallavi	Mysore
11	Pallavi	Mysore
11	Pallavi	Mysore
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri

14 rows selected.

Step3: display the RowId from studnet2 table

SQL> select ID, Sname, location, rowid from student2;

ID	SNAME	LOCATION	ROWID
-----	-----	-----	-----
10	Santosh	Kalburgi	AAAE+YAAEAAAAHdAAA
10	Santosh	Kalburgi	AAAE+YAAEAAAAHdAAB
10	Santosh	Kalburgi	AAAE+YAAEAAAAHdAAC
10	Santosh	Kalburgi	AAAE+YAAEAAAAHdAAD
11	Pallavi	Mysore	AAAE+YAAEAAAAHdAAE
11	Pallavi	Mysore	AAAE+YAAEAAAAHdAAF
11	Pallavi	Mysore	AAAE+YAAEAAAAHdAAG
11	Pallavi	Mysore	AAAE+YAAEAAAAHdAAH
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAI
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAJ
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAK
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAL
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAM
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAN

14 rows selected.

Step4: display min(rowid) and max(rowid) from student2 table

SQL> select id, sname, location, min(rowid), max(rowid) from student2 group by id, sname, location;

ID	SNAME	LOCATION	MIN(ROWID)	MAX(ROWID)
-----	-----	-----	-----	-----
10	Santosh	Kalburgi	AAAE+YAAEAAAAHdAAA	AAAE+YAAEAAAAHdAAD
11	Pallavi	Mysore	AAAE+YAAEAAAAHdAAE	AAAE+YAAEAAAAHdAAH
12	SantoshUb	Yadgiri	AAAE+YAAEAAAAHdAAI	AAAE+YAAEAAAAHdAAN

Step5: display only duplicate records except 1 set of records

Note: for min

```
SQL> select * from student2 where rowid not in (select min(rowid) from student2 group by id,
sname, location) order by id ASC;
```

ID	SNAME	LOCATION
10	Santosh	Kalburgi
10	Santosh	Kalburgi
10	Santosh	Kalburgi
11	Pallavi	Mysore
11	Pallavi	Mysore
11	Pallavi	Mysore
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri
12	SantoshUb	Yadgiri

11 rows selected.

Note: for max

```
SQL> select * from student2 where rowid not in (select max(rowid) from student2 group by id,
sname, location);
```

ID	SNAME	LOCATION
10	Santosh	Kalburgi
12	SantoshPal	Yadgiri
11	Pallavi	Mysore
11	Pallavi	Mysore
12	SantoshPal	Yadgiri
12	SantoshPal	Yadgiri
11	Pallavi	Mysore
10	Santosh	Kalburgi
12	SantoshPal	Yadgiri
12	SantoshPal	Yadgiri
10	Santosh	Kalburgi

11 rows selected.

Based on the true condition display the records

```
SQL> select * from dept where 1=1;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS

30	SALES	CHICAGO
40	OPERATIONS	BOSTON

SQL> select * from dept where 1=4;

no rows selected

ROW Comparison operator (all, any, some): -

All operator: The All operator applies 'and' logical operator on each value specified in the list

Example;

SQL> select * from emp where sal>=ALL (2000,3000);

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

Or

SQL> select * from emp where sal>=2000 and sal>=3000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

ANY Operator: The ANY operator applies 'OR' logical operators on each value specified in the list

Example;

SQL> select * from emp where sal>=ANY (2000,3000);

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

6 rows selected.

Or

SQL> select * from emp where sal>=2000 OR sal>=3000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

6 rows selected.

SOME Operator: The SOME operator applies 'OR' logical operator on each value specified in the list

Example;

SQL> select * from emp where sal>=SOME (2000,3000);

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

6 rows selected.

Or

SQL> select * from emp where sal>=2000 or sal>=3000;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7902	FORD	ANALYST	7566	03-DEC-81	3000		20

6 rows selected.

Table alias: -

The main purpose of using alias to differentiate column names specific to table wise

SQL> select d.deptno, d.dname, d.loc from dept d;

DEPTNO	DNAME	LOC
-----	-----	-----
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Rule for Table alias:

- The alias name which is used for the table name the same alias name should be used for the specific table column name

Core related sub queries: -

Core related sub queries are also called as nested query, here the inner query and outer query executes parallelly

WAQ to display the nth max salary

```
SQL> select distinct a.sal from emp a
      where &n=
      (select count(distinct(b.sal)) from emp b
      where a.sal<=b.sal;
```

Enter value for n: 2
old 2: where &n=
new 2: where 1=
SAL

5000

Structural representation of above Query:

a.sal	b.sal
7000	7000
8000	8000
9000	9000
6000	6000
5000	5000
4000	4000
7000	7000

If I want to find max salary check 1 by 1 number with all the no from another table which number is going to become 1=1,2=2,3=3, etc. or true, then that record is our max salary

Example;

If I want to check 1st max salary take a.sal of 7000 is <= to b.sal of 7000 then it is true. Then a.sal of 7000 is <= b.sal of 8000, yes then it is also true in this case it won't satisfy our 1st max of a.sal= 1st max of b.sal so then move on to next a.sal number to check 1=1 until condition become true.

Here, 7000<=7000 yes
7000<=8000 yes
7000<=9000 yes
7000<=6000 no here it satisfies 1=3 condition so it's not our 1st max num then go for another a.sal
no i.e.,
8000<=7000 no
8000<=8000 yes
8000<=9000 yes
8000<=6000 no here it satisfies 1=2 condition so it's not our 1st max num then go for another a.sal
no i.e.,
9000<=7000 no
9000<=8000 no
9000<=9000 yes
9000<=6000 no here it satisfies 1=1 condition so it's our 1st max num then no need to go for another a.sal no i.e, so here 9000 is our 1st max number.

WAQ to display the dname in which employees are available

```
SQL> select d.dname from dept d
      where d.deptno = (select distinct e.deptno from emp e
      where d.deptno = e.deptno);
```

DNAME

ACCOUNTING

RESEARCH

SALES

Or

```
SQL> select d.dname from dept d
      where d.deptno in(select e.deptno from emp e
      where d.deptno = e.deptno);
```

DNAME

ACCOUNTING

RESEARCH

SALES

WAQ to display the dname in which employees are not working

```
SQL> select d.dname from dept d
      where d.deptno not in(select e.deptno from emp e
      where d.deptno = e.deptno)
```

DNAME

OPERATIONS

Exists clause: -

Based on the matching condition available the inner query provides records in this case the exists clause provides true and displays the records, otherwise it provides false and it does not display any records i.e., exists {1,2,3}
=> true

Exists {}
=> false

WAQ to display the dname in which employees are working

```
SQL> select d.dname from dept d where exists  
      (select sal from emp e  
       where d.deptno = e.deptno)
```

DNAME

ACCOUNTING
RESEARCH
SALES

WAQ to display the dname in which employees are getting comm

```
SQL> select d.dname from dept d where exists  
      (select * from emp e  
       Where d.deptno = e.deptno  
       And e.comm is not null);
```

DNAME

SALES

WAQ to display the dname in which employees are not getting comm

```
SQL> select d.dname from dept d where exists  
      (select * from emp e  
       where d.deptno = e.deptno  
       and e.comm is null)
```

DNAME

ACCOUNTING
RESEARCH
SALES

WAQ to display in which employees are working and who is getting min salary

```
SQL> select d.dname from dept d where exists
      (select * from emp e
       where d.deptno = e.deptno
       and e.sal = (select min(sal) from emp e))
```

```
DNAME
-----
RESEARCH
```

NOT Exists: -(Inverse operation)

WAQ to display dname in which employees are not working

```
SQL> select d.dname from dept d where not exists
      (select * from emp e
       where d.deptno = e.deptno)
```

```
DNAME
-----
OPERATIONS
```

Note: SQL> ed is called as editor window, when we call this, it provides new window to edit query because in command line we can't move cursor up & down

* To execute that editing query press alt+F4 and save then press / key in SQL command line to execute

Joins: - ***

It is the superior topic

Joins are used to display columns from multiple tables based on the matching condition, in order to perform joins, there must be a common column or common records between the 2 tables

Types of Joins:

- Cartesian join or Cross Join
- Inner join or Equi join
- Left join or left outer join
- Right join or Right outer join
- Full join or Full outer join
- Self-join

Cartesian Join or Cross Join: -

It works based on the principle of cartesian product

Example;

A = {1,2}

B = {3,4,5}

A * B = {(1,3),(1,4),(1,5),(2,3),(2,4),(2,5)}

Or

A * B = {(3,1),(4,1),(5,1),(3,2),(4,2),(5,2)}

SQL> select e.empno, e.ename, e.job, d.deptno, d.dname, d.loc from emp e, dept d;

EMPNO	ENAME	JOB	DEPTNO	DNAME	LOC
-----	-----	-----	-----	-----	-----
7369	SMITH	CLERK	10	ACCOUNTING	NEW YORK
7499	ALLEN	SALESMAN	10	ACCOUNTING	NEW YORK
7521	WARD	SALESMAN	10	ACCOUNTING	NEW YORK
7566	JONES	MANAGER	10	ACCOUNTING	NEW YORK
7654	MARTIN	SALESMAN	10	ACCOUNTING	NEW YORK
7698	BLAKE	MANAGER	10	ACCOUNTING	NEW YORK
7782	CLARK	MANAGER	10	ACCOUNTING	NEW YORK
7788	SCOTT	ANALYST	10	ACCOUNTING	NEW YORK
7839	KING	PRESIDENT	10	ACCOUNTING	NEW YORK
7844	TURNER	SALESMAN	10	ACCOUNTING	NEW YORK
7876	ADAMS	CLERK	10	ACCOUNTING	NEW YORK
7900	JAMES	CLERK	10	ACCOUNTING	NEW YORK
7902	FORD	ANALYST	10	ACCOUNTING	NEW YORK
7934	MILLER	CLERK	10	ACCOUNTING	NEW YORK
7369	SMITH	CLERK	20	RESEARCH	DALLAS
7499	ALLEN	SALESMAN	20	RESEARCH	DALLAS
7521	WARD	SALESMAN	20	RESEARCH	DALLAS
7566	JONES	MANAGER	20	RESEARCH	DALLAS
7654	MARTIN	SALESMAN	20	RESEARCH	DALLAS
7698	BLAKE	MANAGER	20	RESEARCH	DALLAS
7782	CLARK	MANAGER	20	RESEARCH	DALLAS
7788	SCOTT	ANALYST	20	RESEARCH	DALLAS
7839	KING	PRESIDENT	20	RESEARCH	DALLAS
7844	TURNER	SALESMAN	20	RESEARCH	DALLAS
7876	ADAMS	CLERK	20	RESEARCH	DALLAS
7900	JAMES	CLERK	20	RESEARCH	DALLAS
7902	FORD	ANALYST	20	RESEARCH	DALLAS
7934	MILLER	CLERK	20	RESEARCH	DALLAS
7369	SMITH	CLERK	30	SALES	CHICAGO
7499	ALLEN	SALESMAN	30	SALES	CHICAGO
7521	WARD	SALESMAN	30	SALES	CHICAGO
7566	JONES	MANAGER	30	SALES	CHICAGO
7654	MARTIN	SALESMAN	30	SALES	CHICAGO
7698	BLAKE	MANAGER	30	SALES	CHICAGO
7782	CLARK	MANAGER	30	SALES	CHICAGO
7788	SCOTT	ANALYST	30	SALES	CHICAGO
7839	KING	PRESIDENT	30	SALES	CHICAGO
7844	TURNER	SALESMAN	30	SALES	CHICAGO
7876	ADAMS	CLERK	30	SALES	CHICAGO
7900	JAMES	CLERK	30	SALES	CHICAGO
7902	FORD	ANALYST	30	SALES	CHICAGO
7934	MILLER	CLERK	30	SALES	CHICAGO
7369	SMITH	CLERK	40	OPERATIONS	BOSTON

7499	ALLEN	SALESMAN	40	OPERATIONS	BOSTON
7521	WARD	SALESMAN	40	OPERATIONS	BOSTON
7566	JONES	MANAGER	40	OPERATIONS	BOSTON
7654	MARTIN	SALESMAN	40	OPERATIONS	BOSTON
7698	BLAKE	MANAGER	40	OPERATIONS	BOSTON
7782	CLARK	MANAGER	40	OPERATIONS	BOSTON
7788	SCOTT	ANALYST	40	OPERATIONS	BOSTON
7839	KING	PRESIDENT	40	OPERATIONS	BOSTON
7844	TURNER	SALESMAN	40	OPERATIONS	BOSTON
7876	ADAMS	CLERK	40	OPERATIONS	BOSTON
7900	JAMES	CLERK	40	OPERATIONS	BOSTON
7902	FORD	ANALYST	40	OPERATIONS	BOSTON
7934	MILLER	CLERK	40	OPERATIONS	BOSTON

56 rows selected.

Cross Join:

```
SQL> select e.empno, e.ename, e.job, d.deptno, d.dname, d.loc from emp e, dept d;
```

Output.(Same as Cartesian join)

Inner Join or Equi Join:

The inner join or Equi joins returns matching records from both the tables based on the matching condition specified

```
SQL> select * from person;
```

ID	NAME
10	Santosh
20	Shilpa
30	Krishna
40	Vinith
50	Sahana
60	Anusha

6 rows selected.

```
SQL> select * from location;
```

ID	CITY
10	Bangalore
20	Kolar
30	Mysore
40	Bidar
80	Raichur

90	Hubli
40	Gulbarga

7 rows selected.

From table WAQ to display inner join Example

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
50	Sahana	80	Raichur
60	Anusha	90	Hubli
		40	Gulbarga

SQL> select p.id, p.name, loc.id, loc.city from person p, location loc where p.id = loc.id;

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
40	Vinith	40	Gulbarga

Case1: Perform inner join on emp and dept table by applying where clause

WAQ to display employee names and their corresponding dname in which they are working

SQL> select e.ename, d.dname from emp e, dept d
where e.deptno = d.deptno;

ENAME	DNAME
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES

MARTIN SALES

14 rows selected.

Case2: Perform inner join on emp and dept table by applying ON Clause

WAQ to display employee name and their corresponding dname in which they are working

```
SQL> select e.ename, d.dname from emp e INNER JOIN dept d
      ON e.deptno = d.deptno;
```

ENAME	DNAME
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES

14 rows selected.

Left join or Left Outer Join:

Left Join returns all the records from the left table and only the matching records from the right table

Take the data from this table to WAQ

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
50	Sahana	80	Raichur
60	Anusha	90	Hubli
		40	Gulbarga

```
SQL> select p.id, p.name, loc.id, loc.city from person p, location loc
      where p.id = loc.id (+);
```

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
40	Vinith	40	Gulbarga
50	Sahana		
60	Anusha		

7 rows selected.

Case1: Perform left join on emp and dept table by applying Where Clause

WAQ to display all available employee names and their corresponding dname

```
SQL> select e.ename, d.dname from emp e, dept d
      where e.deptno = d.deptno(+);
```

ENAME	DNAME
MILLER	ACCOUNTING
KING	ACCOUNTING
CLARK	ACCOUNTING
FORD	RESEARCH
ADAMS	RESEARCH
SCOTT	RESEARCH
JONES	RESEARCH
SMITH	RESEARCH
JAMES	SALES
TURNER	SALES
BLAKE	SALES
MARTIN	SALES
WARD	SALES
ALLEN	SALES

14 rows selected.

Note: It is similar to looks like inner join only because inner join is the foundation for all the joins

Case2: Perform left join on emp and dept table by applying On Clause

WAQ to display all available employee names and their corresponding dname

```
SQL> select e.ename, d.dname from emp e LEFT JOIN dept d
      ON e.deptno = d.deptno;
```

ENAME	DNAME
-----	-----

MILLER	ACCOUNTING
KING	ACCOUNTING
CLARK	ACCOUNTING
FORD	RESEARCH
ADAMS	RESEARCH
SCOTT	RESEARCH
JONES	RESEARCH
SMITH	RESEARCH
JAMES	SALES
TURNER	SALES
BLAKE	SALES
MARTIN	SALES
WARD	SALES
ALLEN	SALES

14 rows selected.

Note: Here in ON clause no need to use (+) symbol because the name left join will take care of joining the left join concept

Right Join or Right Outer Join:

The Right Outer join returns all the records from right table and only the matching records from left table

Or

It is an inverse to Left Join or Left Outer Join

Take the data from this table to WAQ

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
50	Sahana	80	Raichur
60	Anusha	90	Hubli
		40	Gulbarga

SQL> select p.id, p.name, loc.id, loc.city from person p, location loc
where p.id (+) = loc.id;

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Gulbarga
40	Vinith	40	Bidar
		90	Hubli

7 rows selected.

Case1: Perform Right Join on emp and dept tables by applying Where Clause

WAQ to display employee names and their corresponding dname along with all available dept names

```
SQL> select e.ename, d.dname from emp e, dept d
      where e.deptno (+) = d.deptno;
```

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES
	OPERATIONS

15 rows selected.

Case2: Perform Right Join on emp and dept tables by applying ON Clause

WAQ to display employee names and their corresponding dname along with all available dname

```
SQL> select e.ename, d.dname from emp e RIGHT JOIN dept d
      on e.deptno = d.deptno;
```

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES

TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES
	OPERATIONS

15 rows selected.

Full Join or Full Outer Join:

The Full Join return matching as well as non-matching records from both tables

Or

It is a Combination of both Left Join and Right Join

Take the date from this table to WAQ

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
50	Sahana	80	Raichur
60	Anusha	90	Hubli
		40	Gulbarga

SQL> select p.id, p.name, loc.id, loc.city from person p FULL JOIN location loc
ON p.id = loc.id;

ID	NAME	ID	CITY
10	Santosh	10	Bangalore
20	Shilpa	20	Kolar
30	Krishna	30	Mysore
40	Vinith	40	Bidar
		80	Raichur
		90	Hubli
40	Vinith	40	Gulbarga
50	Sahana		
60	Anusha		

9 rows selected.

Case1: Perform Full Join on emp and dept table by applying Where Clause

WAQ to display all available employee name, job, sal and their corresponding dname and location along with all available dname, location

```
SQL> select e.ename, e.job, e.sal, d.dname, d.loc from emp e, dept d
      where e.deptno = d.deptno(+)
      Union
      select e.ename, e.job, e.sal, d.dname, d.loc from emp e, dept d
      where e.deptno(+) = d.deptno
```

ENAME	JOB	SAL	DNAME	LOC
-----	-----	-----	-----	-----
ADAMS	CLERK	1100	RESEARCH	DALLAS
ALLEN	SALESMAN	1600	SALES	CHICAGO
BLAKE	MANAGER	2850	SALES	CHICAGO
CLARK	MANAGER	2450	ACCOUNTING	NEW YORK
FORD	ANALYST	3000	RESEARCH	DALLAS
JAMES	CLERK	950	SALES	CHICAGO
JONES	MANAGER	2975	RESEARCH	DALLAS
KING	PRESIDENT	5000	ACCOUNTING	NEW YORK
MARTIN	SALESMAN	1250	SALES	CHICAGO
MILLER	CLERK	1300	ACCOUNTING	NEW YORK
SCOTT	ANALYST	3000	RESEARCH	DALLAS
SMITH	CLERK	800	RESEARCH	DALLAS
TURNER	SALESMAN	1500	SALES	CHICAGO
WARD	SALESMAN	1250	SALES	CHICAGO
			OPERATIONS	BOSTON

15 rows selected.

Case2: Perform Full Join on emp and dept table by applying ON Clause

WAQ to display all available employee name, job, sal and their corresponding dname and location along with all available dname and location

```
SQL> select e.ename, e.job, e.sal, d.dname, d.loc from emp e FULL JOIN dept d
      ON e.deptno = d.deptno;
```

ENAME	JOB	SAL	DNAME	LOC
-----	-----	-----	-----	-----
SMITH	CLERK	800	RESEARCH	DALLAS
ALLEN	SALESMAN	1600	SALES	CHICAGO
WARD	SALESMAN	1250	SALES	CHICAGO
JONES	MANAGER	2975	RESEARCH	DALLAS
MARTIN	SALESMAN	1250	SALES	CHICAGO
BLAKE	MANAGER	2850	SALES	CHICAGO
CLARK	MANAGER	2450	ACCOUNTING	NEW YORK

SCOTT	ANALYST	3000	RESEARCH	DALLAS
KING	PRESIDENT	5000	ACCOUNTING	NEW YORK
TURNER	SALESMAN	1500	SALES	CHICAGO
ADAMS	CLERK	1100	RESEARCH	DALLAS
JAMES	CLERK	950	SALES	CHICAGO
FORD	ANALYST	3000	RESEARCH	DALLAS
MILLER	CLERK	1300	ACCOUNTING	NEW YORK
			OPERATIONS	BOSTON

15 rows selected.

Self-Join:

Joining the same table itself by applying Inner Join or Left Join or Right Join etc is called self-join

WAQ to display employee names and their respective reporting manager names

```
SQL> select e.ename "Employee", m.ename "Manager" from emp e, emp m
      where e.mgr = m.empno;
```

Employee	Manager
-----	-----
FORD	JONES
SCOTT	JONES
JAMES	BLAKE
TURNER	BLAKE
MARTIN	BLAKE
WARD	BLAKE
ALLEN	BLAKE
MILLER	CLARK
ADAMS	SCOTT
CLARK	KING
BLAKE	KING
JONES	KING
SMITH	FORD

13 rows selected.

In the above query the employee's name King is missing because he is not reporting to any manager in order to display all employee names, we have to perform self-join with left join

```
SQL> select e.ename "Employee", m.ename "Manager" from emp e, emp m
      where e.mgr = m.empno(+);
```

Employee	Manager
-----	-----
FORD	JONES
SCOTT	JONES
JAMES	BLAKE

TURNER	BLAKE
MARTIN	BLAKE
WARD	BLAKE
ALLEN	BLAKE
MILLER	CLARK
ADAMS	SCOTT
CLARK	KING
BLAKE	KING
JONES	KING
SMITH	FORD
KING	

14 rows selected.

Natural Join: -

 Natural join returns matching records from both the table it works as similar to inner join/ it is another name for inner join

Natural join works based on the following rule:

- Natural Join doesn't require matching condition
- Common column names between the 2 tables along with alias name we should not specify in a select statement, but without alias name we can specify in select statement

WAQ to display employee names and their corresponding dname in which they are working

SQL> select e.ename, d.dname from emp e NATURAL JOIN dept d;

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES

14 rows selected.

Implicit Join: -

While joining 2 or more tables if we are using Where Clause for matching condition that indicates the concept of Implicit Join

WAQ to display employee names and their corresponding dname in which they are working

```
SQL> select e.ename, d.dname from emp e, dept d
      where e.deptno = d.deptno;
```

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES

14 rows selected.

Explicit Join: -

While joining 2 or more tables if we are using Left Join or Right Join or Inner Join etc. keywords along with ON Clause for matching condition that indicates Explicit Joining

WAQ to display employee names and the corresponding dname in which they are working

```
SQL> select e.ename, d.dname from emp e INNER JOIN dept d
      ON e.deptno = d.deptno;
```

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH

SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES

14 rows selected.

Difference between Implicit Join and Explicit Join:

Implicit	Explicit
* While joining 2 or more tables if we are using Where clause that indicates Implicit Join	* While joining 2 or more tables if we are using keywords such as Inner join/LEFT Join /Right join etc along with ON clause that indicates Explicit Join
* select e.ename, d.dname from emp e, dept d Where e.deptno = d.deptno;	* select e.ename, d.dname from emp e INNER join dept d On e.deptno = d.deptno;

Ansi Join: -

While joining 2 or more tables if we are using Left Join or Right Join or Inner Join etc. Keywords along with ON Clause for matching condition that indicates Ansi Join

WAQ to display employee names and their corresponding dname in which they are working

```
SQL> select e.ename, d.dname from emp e INNER JOIN dept d
      ON e.deptno = d.deptno;
```

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES

BLAKE	SALES
MARTIN	SALES

14 rows selected.

Perform Join operation by 'USING Clause':

While joining 2 or more tables if we can use keywords INNER Join or LEFT Join or RIGHT Join etc. along with USING Clause that indicates Explicit Join

Rules Applicable for USING Clause:

- USING Clause never accepts matching condition
- Common column name between 2 tables without alias name should be specified in USING Clause in order to represents matching condition
- Common column name between the 2 tables along with alias name we shouldn't specify in a select statement but without alias name we can specify in select statement

WAQ to display employee name and their corresponding dname in which they are working

```
SQL> select e.ename, d.dname from emp e inner join dept d
      USING (deptno);
```

ENAME	DNAME
-----	-----
CLARK	ACCOUNTING
KING	ACCOUNTING
MILLER	ACCOUNTING
JONES	RESEARCH
FORD	RESEARCH
ADAMS	RESEARCH
SMITH	RESEARCH
SCOTT	RESEARCH
WARD	SALES
TURNER	SALES
ALLEN	SALES
JAMES	SALES
BLAKE	SALES
MARTIN	SALES

14 rows selected.

How to join 3 tables?

Create a new table insurance

```
SQL> select * from insurance;
```

DEPTNO	ITYPE	I PROVIDER	POLICYNO	STARTDATE	ENDDATE
-----	-----	-----	-----	-----	-----

10	Health	TATAAGE	1111	31-JAN-22	13-SEP-23
20	Education	HDFC BANK	2222	08-OCT-22	08-SEP-23
30	Vehicle	Hero HOND	3333	16-JAN-23	03-SEP-23
40	Savings	ICICI	4444	20-SEP-22	18-SEP-23

Using Implicit joining WAQ to display employee names and their corresponding dname in which they are working and their corresponding insurance type and policy number

Using emp, dept and insurance table we can write a query

```
SQL> select e.ename, d.dname, ins.itype, ins.policyno from emp e, dept d, insurance ins
      where e.deptno = d.deptno
      and d.deptno = ins.deptno;
```

ENAME	DNAME	ITYPE	POLICYNO
-----	-----	-----	-----
SMITH	RESEARCH	Education	2222
ALLEN	SALES	Vehicle	3333
WARD	SALES	Vehicle	3333
JONES	RESEARCH	Education	2222
MARTIN	SALES	Vehicle	3333
BLAKE	SALES	Vehicle	3333
CLARK	ACCOUNTING	Health	1111
SCOTT	RESEARCH	Education	2222
KING	ACCOUNTING	Health	1111
TURNER	SALES	Vehicle	3333
ADAMS	RESEARCH	Education	2222
JAMES	SALES	Vehicle	3333
FORD	RESEARCH	Education	2222
MILLER	ACCOUNTING	Health	1111

14 rows selected.

Using Explicit joining WAQ to display employee names and their corresponding dname in which they are working and their corresponding insurance type and policy number

```
SQL> select e.ename, d.dname, ins.itype, ins.policyno from emp e INNER JOIN dept d
      ON e.deptno = d.deptno
      INNER JOIN insurance ins
      ON d.deptno = ins.deptno;
```

ENAME	DNAME	ITYPE	POLICYNO
-----	-----	-----	-----
SMITH	RESEARCH	Education	2222
ALLEN	SALES	Vehicle	3333
WARD	SALES	Vehicle	3333
JONES	RESEARCH	Education	2222
MARTIN	SALES	Vehicle	3333
BLAKE	SALES	Vehicle	3333

CLARK	ACCOUNTING	Health	1111
SCOTT	RESEARCH	Education	2222
KING	ACCOUNTING	Health	1111
TURNER	SALES	Vehicle	3333
ADAMS	RESEARCH	Education	2222
JAMES	SALES	Vehicle	3333
FORD	RESEARCH	Education	2222
MILLER	ACCOUNTING	Health	1111

14 rows selected.

How to join 3 tables by applying using clause?

WAQ to display employee names and their corresponding dname in which they are working and their corresponding insurance type and policy number

```
SQL> select e.ename, d.dname, ins.itype, ins.policyno from emp e INNER JOIN dept d
      USING (deptno)
      INNER JOIN insurance ins
      USING (deptno);
```

ENAME	DNAME	ITYPE	POLICYNO
-----	-----	-----	-----
SMITH	RESEARCH	Education	2222
ALLEN	SALES	Vehicle	3333
WARD	SALES	Vehicle	3333
JONES	RESEARCH	Education	2222
MARTIN	SALES	Vehicle	3333
BLAKE	SALES	Vehicle	3333
CLARK	ACCOUNTING	Health	1111
SCOTT	RESEARCH	Education	2222
KING	ACCOUNTING	Health	1111
TURNER	SALES	Vehicle	3333
ADAMS	RESEARCH	Education	2222
JAMES	SALES	Vehicle	3333
FORD	RESEARCH	Education	2222
MILLER	ACCOUNTING	Health	1111

14 rows selected.

WAQ to display employee names and their corresponding dname in which they are working along with all available insurance type and insurance provider

```
SQL> select e.ename, d.dname, ins.itype, ins.iprovider from emp e INNER JOIN dept d
      On e.deptno = d.deptno
      RIGHT JOIN insurance ins
      ON d.deptno = ins.deptno;
```

ENAME	DNAME	ITYPE	I PROVIDER
-----	-----	-----	-----
CLARK	ACCOUNTING	Health	TATAAGE
KING	ACCOUNTING	Health	TATAAGE
MILLER	ACCOUNTING	Health	TATAAGE
JONES	RESEARCH	Education	HDFC BANK
FORD	RESEARCH	Education	HDFC BANK
ADAMS	RESEARCH	Education	HDFC BANK
SMITH	RESEARCH	Education	HDFC BANK
SCOTT	RESEARCH	Education	HDFC BANK
WARD	SALES	Vehicle	Hero HOND
TURNER	SALES	Vehicle	Hero HOND
ALLEN	SALES	Vehicle	Hero HOND
JAMES	SALES	Vehicle	Hero HOND
BLAKE	SALES	Vehicle	Hero HOND
MARTIN	SALES	Vehicle	Hero HOND
	Savings		ICICI

15 rows selected.

IN Explicit Join can we use Where Clause to filter the records?

Yes, in explicit join we can use where clause to filter the records

```
SQL> select e.ename, d.dname from emp e INNER JOIN dept d
      ON e.deptno = d.deptno
      Where e.job ='SALESMAN';
```

ENAME	DNAME
-----	-----
ALLEN	SALES
TURNER	SALES
MARTIN	SALES
WARD	SALES

VIEWS: -

Views are virtual tables the complex query output can store it in a view

Assume if we can join more than 5 tables each table has millions of records in this case it takes more than 10min to display the output, to resolve this particular challenge the above-mentioned query output we can store it in a view.

How to create a VIEW?

```
SQL> create VIEW vw_emp as select e.ename, d.dname from emp e INNER JOIN dept d
      ON e.deptno = d.deptno;
```


How to drop a VIEW?

```
SQL> drop view vw_emp;
```

What is TNS?

Ans: TNS stands for transparent network substrate, the TNS name contains Oracle server database details based on the TNS name, the user can connect to Oracle database server

In order to create TNS name the following parameters are required

- Host name/IP address of the server machine
- Portable (for Oracle server the portable is always 1521)
- Service name: It is a user defined name

Note:

1) How to get Host name?

Ans: In command prompt type hostname and press enter to display the Hostname (Hostname is nothing but computer name), Hostname is always unique.

2) How to get the IP address?

Ans: In command prompt type Ipconfig and press enter to display the IP address
IPv4 address is nothing but IP address

3) How to connect TNS name?

Ans: TNS name can be created in 2 approaches

- Update TNSnames.ORA file
- NET configuration assistant wizard

Update TNSnames.ORA file:

In oracle installed path navigate to below folder path

C:\oracle\app\oracle\product\11.2.0\server\network\admin

This folder contains file called TNSnames.ORA we can update IP address, Port number and service name to create a new TNS name

Decode Character:

Decode is a character function it works based on the principle of switch statement

```
SQL> select decode (deptno,10,'Ten',20,'Twenty',30,'Thirty', deptno), dname, loc from dept;
```

DECODE (DEPTNO,10,'TEN',20,'TWENTY',30,'T	DNAME	LOC
-----	-----	-----
Ten	ACCOUNTING	NEW YORK
Twenty	RESEARCH	DALLAS
Thirty	SALES	CHICAGO
40	OPERATIONS	BOSTON

NVL: -

The NVL function replaces null value with another value based on the datatype of the column

Example;

```
SQL> select ename, NVL (comm, '0') from emp;
```

ENAME	NVL(COMM, '0')
-----	-----
SMITH	0
ALLEN	300
WARD	500
JONES	0
MARTIN	1400
BLAKE	0
CLARK	0
SCOTT	0
KING	0
TURNER	0
ADAMS	0
JAMES	0
FORD	0
MILLER	0

14 rows selected.

CASE statement: -

```
SQL> select CASE
      when deptno<=30 then 'Employee Available'
      ELSE 'Employee Not Available'
      End, dname, loc from dept;
```

CASEWHENDEPTNO<=30THEN	DNAME	LOC
-----	-----	-----
Employee Available	ACCOUNTING	NEW YORK
Employee Available	RESEARCH	DALLAS
Employee Available	SALES	CHICAGO
Employee Not Available	OPERATIONS	BOSTON

